

Assignment 4: Signal Handling

Operating System Lab (Unix)

Problem Statement

Write C programs to demonstrate signal handling mechanisms in Unix/Linux. The objective is to understand how processes respond to asynchronous events using signals and how signal handling can be customized.

Tasks to Implement

1. Write a program to handle **SIGINT** (Ctrl + C) using **signal()** and display a custom message
2. Modify the above program to ignore **SIGINT**
3. Write a program to handle multiple signals (e.g., **SIGINT**, **SIGTERM**)
4. Write a program to send a signal using **kill()** system call
5. Write a program to demonstrate **alarm()** and **SIGALRM**
6. Write a program to block and unblock signals using **sigprocmask()**
7. Write a program using **sigaction()** instead of **signal()** for robust signal handling

Instructions

- Use C programming language
- Compile using `gcc filename.c -o output`
- Execute using `./output`
- Use headers: **signal.h**, **unistd.h**, **sys/types.h**
- Use infinite loops (**while(1)**) where needed to keep process running
- Use **sleep()** to observe behavior clearly

Sample Output

Example (SIGINT Handling):

```
Press Ctrl + C to test...  
Caught SIGINT signal!
```

Example (alarm):

```
Alarm set for 5 seconds...  
Alarm signal received!
```

Constraints

- Programs must demonstrate actual signal behavior
- Avoid unsafe operations inside signal handlers (e.g., printf misuse)
- Ensure proper use of system calls
- Do not terminate abruptly unless intended

Evaluation Criteria

- Correct Signal Handling – 40%
- Understanding of Concepts – 20%
- Code Quality and Safety – 20%
- Use of System Calls – 10%
- Documentation / Comments – 10%

Submission Guidelines

- Submit all programs in a single compressed (.zip) file
- Folder name: RollNo_Assignment4
- Include source files and sample outputs
- Include a README explaining signal behavior in each program

Bonus Tasks (Optional but Recommended)

1. Write a program to implement a simple timer using signals
2. Demonstrate communication between two processes using signals
3. Handle `SIGCHLD` to manage child processes
4. Create a program that safely terminates on receiving `SIGTERM`
5. Compare `signal()` vs `sigaction()` behavior